ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BLM4522 Ağ Tabanlı Paralel Dağıtım Sistemleri

Proje 5: Veri Temizleme ve ETL Süreçleri Tasarımı
Proje 2: Veritabanı Yedekleme ve Felaketten Kurtarma Planı

Kullanılan Veri Seti: Kaggle – Stroke Prediction Dataset

(https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset)

21.04.2026

Proje Dokümantasyonu

Yavuz Selim Demirhan
22290565

2025–2026 Bahar Dönemi

İÇİNDEKİLER

PROJE 5: VERİ TEMİZLEME VE ETL SÜREÇLERİ TASARIMI

1. Giriş
2. Kullanılan Veri Seti ve Çalışma Ortamı
3. ETL Mimarisi – Extract (Veri Çıkartma)
4. ETL Mimarisi – Transform (Veri Temizleme ve Dönüştürme)
5. ETL Mimarisi – Load (Veri Yükleme)
6. ETL Çıktısı ve Doğrulama Raporu
7. Karşılaşılan Problemler ve Çözümler
8. Sonuç ve Değerlendirme

PROJE 2: VERİTABANI YEDEKLEME VE FELAKETTEN KURTARMA PLANI

1. Giriş
2. Çalışma Ortamı ve Kullanılan Veritabanı
3. Yedekleme Stratejisi
4. Felaket Senaryosu Tasarımı
5. Python İle Felaket Kurtarma Simülasyonu
6. Geri Yükleme Sonrası Doğrulama
7. Sonuç

PROJE 5: VERİ TEMİZLEME VE ETL SÜREÇLERİ TASARIMI

1. GİRİŞ

Bu projede, sağlık verileri alanındaki en kritik mühendislik süreçlerinden biri olan ETL (Extract, Transform, Load) mimarisi tasarlanmış ve uygulanmıştır. Modern hastane sistemlerinde ham verilerin doğrudan kullanılması; eksik değerler, tutarsız formatlar ve analitik değeri bulunmayan kolonlar nedeniyle sistem güvenilirliğini ciddi biçimde tehdit etmektedir. Bu çalışmada, Kaggle platformundan elde edilen gerçek dünya 'Stroke Prediction' (İnme Riski) veri seti kullanılmış; veriler önce ham hâliyle sisteme alınmış, ardından kapsamlı bir temizleme ve dönüştürme sürecinden geçirilmiş ve son olarak güvenilir bir SQLite veritabanına aktarılmıştır.

Projenin temel amacı; 5.110 satırlık hasta kaydından oluşan bu gerçek veri setini makine öğrenmesi ve analiz için kullanılabilir bir yapıya dönüştürmek ve bunun yanı sıra bu temizlenmiş veritabanını Proje 2'deki yedekleme sistemiyle entegre ederek uçtan uca bir veri pipeline'ı oluşturmaktır.

2. KULLANILAN VERİ SETİ VE ÇALIŞMA ORTAMI

Bu projede kullanılan veri seti, Kaggle üzerinden erişilebilen ve tıp dünyasında inme riskini belirlemek amacıyla yaygın şekilde kullanılan 'Stroke Prediction Dataset' (Fedesoriano, 2021) adlı açık kaynak veri setidir. Veri seti şu alanları içermektedir:

- Hasta_ID – Benzersiz hasta tanımlayıcısı (analitik değeri yoktur, silindi)
- Cinsiyet – Erkek / Kadın / Diğer
- Yas – Hastanın yaşı (eksik değerler mevcuttu)
- Hipertansiyon – 0/1 binary değer
- Kalp_Hastaligi – 0/1 binary değer
- Evli_Mi – Evet/Hayır
- Is_Turu – Devlet/Özel/Kendi İşi vb.
- Ikamet_Turu – Şehir/Kırsal
- Ort_Glikoz – Ortalama kan şekeri düzeyi
- Vucut_Kitle_Endeksi – BMI (eksik değerler mevcuttu)
- Sigara_Durumu – İçiyor/İçmedi/Bıraktı vb. (gereksiz sigorta kolonu kaldırıldı)
- Inme_Gecirdi – Hedef değişken (0/1)

Veri 5.110 kayıt ve 12 sütundan oluşmakta olup toplam boyutu yaklaşık 341 KB'tır. Çalışma ortamı olarak Python 3.x, Pandas kütüphanesi ve SQLite veritabanı kullanılmıştır.

3. ETL MİMARİSİ – EXTRACT (VERİ ÇIKARTMA)

ETL sürecinin ilk aşaması olan Extract (Çıkartma) aşamasında, ham hasta kayıtları sisteme alınmıştır. Bu aşama için Pandas kütüphanesinin pd.read_csv() fonksiyonu kullanılmış ve CSV dosyası bellek üzerinde bir DataFrame nesnesine yüklenmiştir. Söz konusu fonksiyon

5.110 satırlık veriyi saniyeler içinde RAM'e aktararak sonraki dönüştürme işlemleri için hazır hâle getirmiştir.

Aşağıda Extract aşamasına ait kod bloğu verilmektedir:

```
import pandas as pd
import sqlite3
import os

CSV_DOSYASI = "../hasta_kayitlari.csv"
HEDEF_DB    = "./Hastane_Temiz_Veri.db"
RAPOR_DOSYASI = "./Hastane_ETL_Raporu.txt"

def hastane_etl_run():
    rapor_satirlari = ["=== HASTANE VERİ TEMİZLEME VE ETL RAPORU ==="]

    # 1. EXTRACT (Veriyi Sistemden Çekme)
    try:
        df = pd.read_csv(CSV_DOSYASI)
        rapor_satirlari.append(
            f"Aşama 1: Ham hasta dosyası yüklendi. İşlem görecek kayıt:
{len(df)}"
        )
    except FileNotFoundError:
        print("hasta verisi bulunamadı!")
        return
```

Bu aşama sonucunda sistemde 5.110 adet kayıt başarıyla tanımlanmış ve Transform aşamasına hazır hâle getirilmiştir.

4. ETL MİMARİSİ – TRANSFORM (VERİ TEMİZLEME VE DÖNÜŞTÜRME)

Transform aşaması, veri kalitesini artırmak amacıyla gerçekleştirilen tüm temizleme, doldurma ve standardizasyon işlemlerini kapsamaktadır. Bu projede üç temel dönüştürme kategorisi uygulanmıştır.

4.1. Eksik Değerlerin Medyan ile Doldurulması

Veri setinde incelendiğinde, 409 adet kayıtta Yaş bilgisinin ve 201 adet kayıtta Vücut Kitle Endeksi (BMI) bilgisinin eksik olduğu tespit edilmiştir. Eksik kayıtlar çıkarılmak yerine, istatistiksel açıdan sapma yaratmayan medyan değerleriyle doldurulmuştur. Ortalama yerine medyanın tercih edilmesinin temel nedeni; veri setinde 1 yaşındaki bebeklerden 85 yaşındaki yetişkinlere kadar geniş bir yaş aralığının bulunmasıdır. Bu aralıkta aritmetik ortalama ciddi sapmalara neden olabilmektedir.

```
# Yaş sütunundaki eksikleri medyan (ortanca) ile doldur
eksik_yas_sayisi = df['Yas'].isnull().sum()
df['Yas'] = df['Yas'].fillna(df['Yas'].median())

# Vücut Kitle Endeksi eksiklerini medyan ile doldur
```

```
eksik_bmi_sayisi = df['Vucut_Kitle_Endeksi'].isnull().sum()
df['Vucut_Kitle_Endeksi'] = df['Vucut_Kitle_Endeksi'].fillna(
    df['Vucut_Kitle_Endeksi'].median()
)
```

4.2. Gereksiz Kolonların Silinmesi

Analitik değeri bulunmayan Hasta_ID kolonu ile model eğitimine katkısı olmayan Sigorta_Gereksiz kolonu, DataFrame'den tamamen kaldırılmıştır. Bu işlem pandas'ın drop() fonksiyonu ile gerçekleştirilmiştir.

```
# Gereksiz kolonları sil
df.drop(
    ['Sigorta_Gereksiz', 'Hasta_ID'],
    axis=1,
    inplace=True,
    errors='ignore'
)
```

4.3. Metin Standardizasyonu

Cinsiyet sütununda farklı kullanıcılar tarafından büyük/küçük harf karışık ve baştaki/sondaki boşluk karakterleriyle birlikte girilen değerler tespit edilmiştir. Bu değerler strip() ve capitalize() fonksiyonlarının kombinasyonu ile standart bir formata dönüştürülmüştür.

```
# Cinsiyet sütununu standardize et
df['Cinsiyet'] = df['Cinsiyet'].str.strip().str.capitalize()
```

5. ETL MİMARİSİ – LOAD (VERİ YÜKLEME)

Tüm temizleme işlemleri tamamlandıktan sonra, bellek üzerindeki DataFrame nesnesi SQLite veritabanına aktarılmıştır. SQLite seçiminin temel gerekçesi; ek bir sunucu kurulumu gerektirmemesi, Python'ın dahili sqlite3 kütüphanesiyle doğrudan entegre çalışması ve Proje 2 kapsamındaki yedekleme sisteminin bu formatı desteklemesidir.

```
# 3. LOAD (Temiz Veriyi SQLite ile Saklama)
if os.path.exists(HEDEF_DB):
    os.remove(HEDEF_DB)  # Önceki veritabanını temizle

hastane_baglanti = sqlite3.connect(HEDEF_DB)

# DataFrame'i 'TemizlenenHastalar' tablosu olarak veritabanına yaz
df.to_sql('TemizlenenHastalar', hastane_baglanti,
          if_exists='replace', index=False)
hastane_baglanti.close()

rapor_satirlari.append(
    f"Aşama 3 (Yükleme): Tüm işlemler tamamlandı. "
    f"Veriler yeni oluşturulan '{HEDEF_DB}' sunucusuna başarıyla taşındı."
```

```
)

# ETL Raporunu dosyaya yaz
with open(RAPOR_DOSYASI, "w", encoding="utf-8") as dosya:
    dosya.write("\n".join(rapor_satirlari))
```

Bu işlem sonucunda 5.110 satırlık temizlenmiş hasta verisi, Hastane_Temiz_Veri.db adlı SQLite veritabanındaki TemizlenenHastalar tablosuna başarıyla aktarılmıştır.

6. ETL ÇIKTISI VE DOĞRULAMA RAPORU

ETL sürecinin tamamlanmasının ardından sistem, işlemlerin özetini içeren bir metin raporu otomatik olarak oluşturmuştur. Aşağıda bu raporun içeriği verilmektedir:

```
=== HASTANE VERİ TEMİZLEME VE ETL RAPORU ===
Aşama 1: Ham hasta dosyası yüklendi. İşlem görecek kayıt: 5110
Aşama 2 (Silme): Çöplük niteliğindeki Sigorta_Gereksiz kolonu ve
                 referans olan Hasta_ID veriden tamamen silindi.
Aşama 2 (Düzeltme): Model eğitimini bozan 409 kayıp Yaş ve
                    201 kayıp Vücut Kitle Endeksi başarıyla onarıldı.
Aşama 2 (Standardizasyon): Dağınık girilen Cinsiyet ibareleri
                           (büyük/küçük harf, gereksiz boşluk)
                           tek formata dönüştürüldü.
Son Durum: Veritabanına aktırılacak temiz kayıt adedi: 5110
Aşama 3 (Yükleme): Tüm işlemler tamamlandı. Veriler yeni oluşturulan
                   './Hastane_Temiz_Veri.db' sunucusuna başarıyla taşındı.
```

Rapor incelendiğinde; ham veri kaybı yaşanmadan tüm kayıtların işlendiği, veri bütünlüğünün korunduğu ve son tablonun analiz için tamamen hazır hâle getirildiği görülmektedir.

7. KARŞILAŞILAN PROBLEMLER VE ÇÖZÜMLER

Proje sürecinde aşağıdaki teknik güçlüklerle karşılaşılmış ve her biri için sistematik çözümler geliştirilmiştir:

Eksik Değer Yönetimi

409 adet Yaş ve 201 adet Vücut Kitle Endeksi değerinin eksik olduğu görülmüştür. Bu kayıtların silinmesi veri kaybına yol açacağından, medyan atama stratejisi tercih edilmiş ve veri bütünlüğü korunmuştur.

Karakter Kodlama ve Format Tutarsızlıkları

Cinsiyet sütununda aynı değer için 'erkek', 'Erkek', ' erkek ' gibi farklı yazımların varlığı tespit edilmiştir. strip() ve capitalize() kombinasyonu bu tutarsızlıkları gidermiştir.

Veritabanı Çakışması

ETL betiği her çalıştırıldığında önceki veritabanı dosyası ile çakışma oluşmaktaydı. Bu sorun, yükleme öncesinde os.path.exists() kontrolü ve os.remove() çağrısıyla çözülmüştür.

8. SONUÇ VE DEĞERLENDİRME

Bu projede Kaggle'dan alınan 5.110 satırlık gerçek dünya hasta verisi üzerinde tam kapsamlı bir ETL süreci uygulanmıştır. Extract aşamasında ham veri pandas ile belleğe alınmış; Transform aşamasında eksik değerler statistiksel yöntemlerle doldurulmuş, gereksiz kolonlar kaldırılmış ve metin standartlaştırması gerçekleştirilmiştir. Load aşamasında ise temizlenen veri SQLite veritabanına aktarılmıştır.

Projenin en önemli kazanımlarından biri, oluşturulan Hastane_Temiz_Veri.db dosyasının doğrudan Proje 2 kapsamındaki yedekleme sistemiyle entegre edilmesidir. Bu sayede ETL fabrikasından çıkan temiz veri, otomatik olarak yedekleme pipeline'ına dahil edilmekte ve uçtan uca bir veri güvenliği altyapısı oluşturulmaktadır.

Gelecek çalışmalarda aşağıdaki geliştirmeler yapılabilir:

- Diğer CSV dosyalarının da ETL sürecine dahil edilmesi
- Zamanlanmış görevler ile ETL'nin otomatik tetiklenmesi
- Veri kalite kontrol sorgularının artırılması
- Tableau veya Power BI ile görsel raporlama eklenmesi
- Daha gelişmiş eksik değer tamamlama algoritmalarının denenmesi

Sonuç olarak bu proje kapsamında Python programlama dili ve Pandas kütüphanesi kullanılarak gerçek bir veri temizleme ve ETL uygulaması başarıyla gerçekleştirilmiş, proje hedefleri eksiksiz biçimde yerine getirilmiştir.

PROJE 2: VERİTABANI YEDEKLEME VE FELAKETTEN KURTARMA PLANI

1. GİRİŞ

Bu projede, Proje 5 kapsamında ETL süreçleriyle temizlenip oluşturulan Hastane_Temiz_Veri.db veritabanı üzerinde yedekleme ve felaketten kurtarma senaryosu tasarlanmış ve uygulanmıştır. Veritabanı sistemlerinde veri kaybı; siber saldırılar, kullanıcı hataları, donanım arızaları ve fidye yazılımları gibi tehditlerden kaynaklanabilmektedir. Özellikle sağlık sistemlerinde hasta kaydı kaybının geri dönüşü yoktur.

Bu çalışmada amaç; MS SQL Server standartlarında çok katmanlı yedekleme stratejisini tasarlamak, Python ile felaketi simüle etmek ve ardından geri yükleme sürecini adım adım göstermektir. Proje 5 ile entegrasyon sayesinde uçtan uca bir veri güvenliği pipeline'ı elde edilmiştir.

2. ÇALIŞMA ORTAMI VE KULLANILAN VERİTABANI

Yedekleme senaryosunda kullanılan veritabanı, Proje 5'te oluşturulan Hastane_Temiz_Veri.db dosyasıdır. Bu SQLite veritabanı, ETL sürecinden geçirilmiş 5.110 satırlık temiz hasta kaydını barındıran TemizlenenHastalar tablosunu içermektedir.

Çalışma ortamı şu bileşenlerden oluşmaktadır:

- Python 3.x – Otomasyon ve simülasyon betikleri için
- sqlite3 (dahili) – Veritabanı bağlantı ve doğrulama işlemleri için
- shutil – Dosya kopyalama ve yedek yönetimi için
- MS SQL Server (T-SQL) – Gerçek ortam yedekleme komutları için

3. YEDEKLEME STRATEJİSİ

Bu projede MS SQL Server standartlarını esas alan üç katmanlı bir yedekleme mimarisi tasarlanmıştır. Bu mimari; tam yedek (Full Backup), diferansiyel yedek (Differential Backup) ve işlem günlüğü yedekleme (Transaction Log Backup) bileşenlerini içermektedir.

3.1. Tam Yedek (Full Backup)

Haftalık olarak alınması planlanan tam yedek, tüm veritabanını tek bir bak dosyasına kaydeder. Bu yedek, olası felaket senaryolarında geri yüklemenin başlangıç noktasını oluşturur.

```
-- Tam Yedek (Full Backup) - Haftalık Rutin
BACKUP DATABASE Hastane_Temiz_Veri
TO DISK = 'C:\Yedekler\Hastane_Temiz_Veri_Full.bak'
WITH FORMAT,
     MEDIANAME = 'SaglikSistemi',
     NAME = 'Hastane Veritabani Tam Yedeği';
GO
```

3.2. Diferansiyel Yedek (Differential Backup)

Günlük olarak çalıştırılan diferansiyel yedek, son tam yedekten bu yana değişen kayıtları alır. WITH DIFFERENTIAL parametresi sayesinde yalnızca yeni veriler yedeklenerek zaman ve depolama tasarrufu sağlanır.

```
-- Diferansiyel Yedek – Günlük Değişen Hasta Kayıtları
BACKUP DATABASE Hastane_Temiz_Veri
TO DISK = 'C:\Yedekler\Hastane_Temiz_Veri_Diff.bak'
WITH DIFFERENTIAL,
     NAME = 'Hastane Veritabani Gunluk Fark Yedeği';
GO
```

3.3. İşlem Günlüğü Yedekleme (Transaction Log Backup)

Sistemdeki her işlem, saniye granülerliğinde Transaction Log'a kaydedilmektedir. Bu yapı, Point-in-Time Restore senaryolarında dakika ve saniye bazında geri dönüşü mümkün kılar.

```
-- Transaction Log – Nokta Atışı Hasta Kaydı Güvencesi
BACKUP LOG Hastane_Temiz_Veri
TO DISK = 'C:\Yedekler\Hastane_Temiz_Veri_Log.trn'
WITH NAME = 'Hastane Islem Gunlugu Yedeği';
GO
```

4. FELAKET SENARYOSU TASARIMI

Projede gerçekçi bir felaketten kurtarma sürecini gösterebilmek amacıyla Point-in-Time Restore (Noktasal Zaman Kurtarması) senaryosu tasarlanmıştır. Senaryo şu varsayıma dayanmaktadır: saat 16:00'da sistem çökmüştür; ancak 15:59:59 itibarıyla veriler sağlıklı durumdaydı. Görev, veritabanını bu son sağlıklı noktaya geri getirmektir.

Point-in-Time Restore için T-SQL komutları aşağıda verilmektedir:

```
-- Felaket anındaki kaydı güvence altına al (Tail-Log)
BACKUP LOG Hastane_Temiz_Veri
TO DISK = 'C:\Yedekler\Hastane_Temiz_Veri_TailLog.trn'
WITH NORECOVERY;
GO

-- Adım 1: Tam yedeği yükle (henüz açma)
RESTORE DATABASE Hastane_Temiz_Veri
FROM DISK = 'C:\Yedekler\Hastane_Temiz_Veri_Full.bak'
WITH NORECOVERY;
GO

-- Adım 2: Diferansiyel yedeği ekle
RESTORE DATABASE Hastane_Temiz_Veri
FROM DISK = 'C:\Yedekler\Hastane_Temiz_Veri_Diff.bak'
WITH NORECOVERY;
GO
```

```
-- Adım 3: Tam olarak 15:59:59'da durdur!
RESTORE LOG Hastane_Temiz_Veri
FROM DISK = 'C:\Yedekler\Hastane_Temiz_Veri_Log.trn'
WITH STOPAT = '2023-12-05 15:59:59', RECOVERY;
GO
```

STOPAT parametresi, SQL Server'a 'İşlemleri tara ve tam olarak bu saniyeyi gördüğünde dur' komutunu vermekte, böylece felaket anından yalnızca 1 saniye öncesine zamanda yolculuk yapılmaktadır.

5. PYTHON İLE FELAKET KURTARMA SİMÜLASYONU

Yedekleme senaryosunu somutlaştırmak amacıyla Python ile otomatik bir felaket simülasyonu yazılmıştır. Bu betik; Proje 5 çıktısı olan Hastane_Temiz_Veri.db dosyasını kaynak olarak alır, zaman damgalı bir tam yedeğini oluşturur ve ardından felaketi simüle ederek en güncel yedekten geri yükleme yapar.

```
import sqlite3
import shutil
import time
import os
from datetime import datetime

# Veri yolları - Proje 5 çıktısı kaynak olarak kullanılıyor
KAYNAK      = "../Proje 5 Veri Temizleme ve ETL Süreçleri
Tasarımı/Hastane_Temiz_Veri.db"
YEDEK_DIZINI = "./yedek_deposu"

# Klasör yoksa oluştur
if not os.path.exists(YEDEK_DIZINI):
    os.makedirs(YEDEK_DIZINI)

def tam_yedek_baslat():
    """Tam yedekleme: zaman damgalı kopyayı yedek dizinine taşı."""
    zaman_damgasi = datetime.now().strftime("%Y%m%d_%H%M%S")
    yedek_dosyasi = f"{YEDEK_DIZINI}/Hastane_Full_{zaman_damgasi}.db"
    try:
        shutil.copy2(KAYNAK, yedek_dosyasi)
        print(f"Yedekleme başarılı. Dosya: {yedek_dosyasi}")
    except Exception as e:
        print(f"Yedekleme sırasında hata: {e}")

def felaket_kurtarma_simulasyonu():
    """En güncel yedekten yeni veritabanı oluştur."""
    yedekler = sorted(
        [d for d in os.listdir(YEDEK_DIZINI) if d.endswith(".db")],
        reverse=True
    )
    if not yedekler:
        print("Yüklenebilecek yedek bulunamadı.")
        return

    en_yeni_yedek = os.path.join(YEDEK_DIZINI, yedekler[0])
    yeni_db       = "./Kurtarilan_HastaneDB.db"
```

```
    print(f"Sistem çökmesi simüle ediliyor... Kurtarma başlatıldı:
{en_yeni_yedek}")
    shutil.copy2(en_yeni_yedek, yeni_db)
    print(f"Veriler kurtarıldı. Yeni hedef: {yeni_db}")

    try:
        conn  = sqlite3.connect(yeni_db)
        cur   = conn.cursor()
        cur.execute("SELECT name FROM sqlite_master WHERE type='table';")
        print(f"Bağlantı onayı başarılı. Aktif tablolar: {cur.fetchall()}")
        conn.close()
    except sqlite3.Error as e:
        print(f"Veritabanı kurtarılırken hata: {e}")

if __name__ == "__main__":
    tam_yedek_baslat()
    time.sleep(1)     # 1 saniyelik suni gecikme (felaket anı simülasyonu)
    felaket_kurtarma_simulasyonu()
```

Python betiği çalıştırıldığında aşağıdaki çıktı elde edilmektedir:

```
Yedekleme başarılı. Dosya: yedek_deposu/Hastane_Full_20260421_010001.db
Sistem çökmesi simüle ediliyor... Kurtarma başlatıldı:
yedek_deposu/Hastane_Full_20260421_010001.db
Veriler kurtarıldı. Yeni hedef: ./Kurtarilan_HastaneDB.db
Bağlantı onayı başarılı. Aktif tablolar: [('TemizlenenHastalar',)]
```

Çıktı değerlendirildiğinde; zaman damgalı yedeğin başarıyla oluşturulduğu, felaketin simüle edildiği, yedeğin yeni bir veritabanına aktarıldığı ve SQLite metadata sorgusu ile tablonun sağlıklı biçimde geri geldiği doğrulanmaktadır.

6. GERİ YÜKLEME SONRASI DOĞRULAMA

Kurtarılan veritabanının bütünlüğünü doğrulamak amacıyla SQLite'ın sqlite_master tablosu üzerinden aktif tablolar sorgulanmış ve TemizlenenHastalar tablosunun mevcut olduğu teyit edilmiştir. Yüksek erişilebilirlik için tasarlanan Mirroring (Aynalama) komutu da aşağıda sunulmaktadır:

```
-- Yüksek Erişilebilirlik: Sunucu Aynalama
ALTER DATABASE Hastane_Temiz_Veri
SET PARTNER = 'TCP://BackupServer.saglik.local:5022';
GO

ALTER DATABASE Hastane_Temiz_Veri
SET PARTNER SAFETY FULL;
GO
```

Bu komutlar sayesinde her veri değişikliği anlık olarak pasif yedek sunucusuna da kopyalanmakta ve birincil sunucunun devre dışı kalması durumunda sistemin kesintisiz çalışması güvence altına alınmaktadır.

7. SONUÇ

7.1. Uygulanan Sürecin Değerlendirilmesi

Bu projede hastane veritabanı güvenliği için kapsamlı bir yedekleme ve felaketten kurtarma altyapısı oluşturulmuştur. MS SQL Server standartlarında üç katmanlı yedekleme stratejisi tasarlanmış; Point-in-Time Restore ile saniye bazlı geri dönüş mümkün kılınmış ve Python betiğiyle tüm süreç otomasyon altına alınmıştır.

Projenin en kritik başarısı, yedekleme sistemini Proje 5 ETL pipeline'ıyla entegre etmesidir. Kaggle'dan çekilen ham veri temizlenerek Hastane_Temiz_Veri.db dosyasına yüklenmekte, bu dosya otomatik olarak yedek deposuna kopyalanmakta ve felaket senaryosunda geri yüklenerek sistem hayata döndürülmektedir.

7.2. Gelecek Geliştirme ve Öneriler

Gelecek çalışmalarda aşağıdaki geliştirmeler önerilebilir:

- Zamanlanmış görevler (cron / Windows Task Scheduler) ile otomatik yedekleme
- Diferansiyel ve artımlı yedekleme stratejileri ile depolama optimizasyonu
- Geri yükleme sürelerinin ölçülerek RTO/RPO hedeflerinin belirlenmesi
- Farklı hata senaryolarının (disk hatası, fidye yazılımı) simüle edilmesi
- Bulut depolama entegrasyonu ile coğrafi yedekleme altyapısı

Sonuç olarak bu proje kapsamında Python ve SQL Server komutları kullanılarak gerçek bir veritabanı yedekleme ve felaketten kurtarma uygulaması başarıyla gerçekleştirilmiş; proje hedefleri eksiksiz biçimde yerine getirilmiştir.